PyTypeWizard

An Automated Program Repair Tool for Python Static Type Errors

SE 801 - Software Project Lab 3
Final Report

Submitted by
Istiaq Ahmed Fahad
BSSE Roll No. : 1204
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Zerina Begum
Professor
Institute of Information Technology
University of Dhaka

Institute of Information Technology
University of Dhaka

Letter of Transmittal

January 30, 2025
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - PyTypeWizard: An Automated Program Repair Tool for Python Static Type Error.

Dear Sir,
I am submitting the technical report for my Software Project Lab 3, *PyTypeWizard: An Automated Program Repair Tool for Python Static Type Error*. This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Istiaq Ahmed Fahad
BSSE-1204
Institute of Information Technology
University of Dhaka

Supervisor's Signature

Acknowledgment

I would like to express my sincere gratitude to the Institute of Information Technology, University of Dhaka, for the opportunity to work on the "PyTypeWizard: An Automated Program Repair Tool for Python Static Type Error" project. This journey has been both rewarding and enlightening. A special thanks to my supervisor, Dr. Zerina Begum, Professor at the Institute of Information Technology, whose guidance and encouragement were invaluable throughout the project. Her insights helped shape the project's direction and fueled my commitment to excellence. I also appreciate my classmates' camaraderie and valuable feedback, which added depth to this experience and helped refine the project.

Abstract

Python's dynamically typed nature allows for flexibility but opens the door for type-related errors that could be very hard to detect and debug, especially on large projects. Without type enforcement, such errors could go unnoticed until runtime when hidden bugs and unpredictable behavior would result.

To address these challenges, we propose **PyTypeWizard**, an automated program repair tool that can detect and resolve Python static type errors in real time. As a VSCode extension, PyTypeWizard continuously monitors code for type mismatches and incorrect type hints, offering immediate feedback and context-aware suggestions for corrections using large language models.

This tool closes the gap between Python's dynamic nature and the benefits of static typing by providing clear explanations for detected errors and potential fixes, improving code reliability, improving the developer experience, and enabling better collaboration. The proposed solution will enable developers to create robust code by reducing debugging effort, improving auto-complete suggestions, and making the codebase maintainable in the long run.

Table of Contents

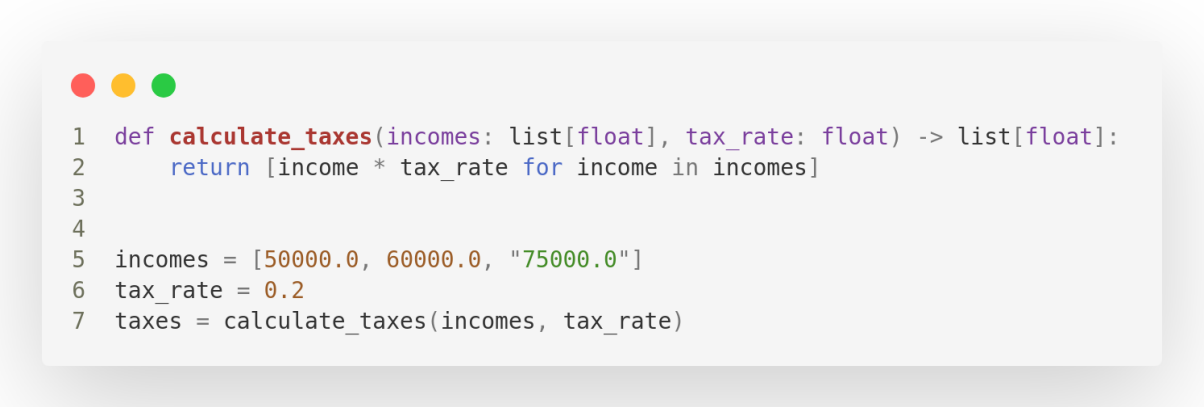
Table of Figures

1. Introduction

Dynamic typing can also be both an asset and a problem. It gives developers the ability to write code quicker, without having to worry about variable types. However, that ease of use may come at a price: in larger projects, undetected type-related errors can hide within the code and cause unpredictable bugs, which are very costly to track down later on. The lack of immediate error detection makes these issues hard to detect before they begin to interfere with the functioning of the program, turning what appears to be a small mistake into a huge issue.

Introducing PyTypeWizard: The automated solution to tame the dynamic nature of Python. This tool acts as a watchful guardian that continuously checks your code and catches type mismatches before they turn into problems. This is how, through the finding and fixing of type errors in real time, PyTypeWizard provides immediate feedback for developers to create cleaner, more reliable code. PyTypeWizard fixes not only existing problems but also guides users toward a better understanding of appropriate type usage, turning pains in coding into learning opportunities.

1.1 Motivation



```python
def calculate_taxes(incomes: list[float], tax_rate: float) -> list[float]:
    return [income * tax_rate for income in incomes]


incomes = [50000.0, 60000.0, "75000.0"]
tax_rate = 0.2
taxes = calculate_taxes(incomes, tax_rate)
```

Figure 1: *Screenshot of Type Annotated Method in Python*

Before getting into the details of my motivation let's look at the example code in Figure 1. Here we can see a function designed to calculate taxes from income salaries. The type hint specifies that the function only accepts a list of floats for the income parameter and a float for tax_rate. Sometimes when we extract data from a CSV file, we may have some numeric values in string format (like 1000 can be “1000”) if the data are not read and processed efficiently. In such cases, the function will fail to calculate the tax correctly. Interestingly in that scenario, Python won’t raise any error when we write the method, allowing the error-prone code to go unnoticed until execution.

Such incorrect type hints can lead to misleading documentation and hidden bugs that become increasingly difficult to debug as the application scales. Additionally, when I interrogated some professional Python Developers about this type of hints of Python, they commented, "*If you consider runtime performance, there is no difference. But real-life software projects are much more than that. No type hinting means other developers will struggle to understand what types a function expects. Also, linters (a tool that analyzes source code for errors, and vulnerabilities) won't detect basic mistakes like trying to access an attribute that doesn't exist. Your editor won't auto-complete stuff for you as well.*"

Inspired by the insights from the papers ***PyTy: Repairing Static Type Errors in Python [1]*** and ***Generative Type Inference for Python [2]*** and valuable feedback from developers, I feel the urge for a tool to address the challenges of type-related issues in Python.

1.2 Project Description

In programming, static typing means that the types of variables are defined clearly, which helps catch errors early and makes the code easier to understand. Python uses dynamic typing, which is more flexible but can cause type-related problems that only show up when the program is run. In addition, Python's dynamic typing lacks strict type enforcement during execution which allows developers to pass incorrect types (like a string instead of a float) without immediate errors. This can lead to hidden bugs, unexpected crashes, and uncertainty about what would be the type of function parameter or variables, making it challenging for developers to understand and maintain the code, especially in larger projects.

To solve this type-related issue, PyTypeWizard, a VSCode extension, is proposed as a tool to automatically detect and fix type-related errors in Python. The goal is to bridge the gap between Python's dynamic typing and the benefits of static type enforcement, helping developers catch and correct type-related issues in runtime.
The core issue we aim to solve is the lack of real-time type error detection and the absence of helpful type suggestions in Python environments. PyTypeWizard will continuously monitor Python code within the VSCode environment, to detect type mismatches or incorrect type hints. By leveraging a large language model (LLM), the tool will not only detect these errors but also suggest corrections based on the code repository which will help to reduce debugging effort.
Additionally, the tool will provide clear explanations about the type-related errors detected and potential ways to fix them. Thus it will help the developer to use correct type hints enabling the IDEs to show auto-complete suggestions. This will not only improve the developer experience but also facilitate better collaboration and long-term maintenance of the codebase, especially in large-scale projects.

1.3 Project Scope

The scope of **PyTypeWizard** is mentioned below:

- It will be a VSCode extension.
- It will detect type errors on the fly.
- A transformer-based model along with an LLM will be used to provide correct suggestions for type errors.
- This extension will support minimum Python version 3.x.x.

1.4 Timeline

The estimated timeline for PyTypeWizard is delineated below:

Figure 2: *Project Timeline*

1.5 Purpose of Document

The purposes of this document are:

- Identify and analyze the requirements
- Design the architecture
- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

This document outlines the Quality Function Deployment (QFD) process for PyTypeWizard. The development of PyTypeWizard's QFD began with an extensive review of relevant research, drawing significant insights from the work of Chow, Grazia, and Pradel [1]. Their findings offered a deep understanding of the domain and informed our approach.

In addition to this research, we engaged directly with key stakeholders from the software development industry. By consulting with developers across various organizations, we gained valuable perspectives on their needs and challenges, ensuring that PyTypeWizard addresses real-world concerns effectively.

2.1 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. For this project, the following requirements are identified-

2.2.1 Normal Requirements

These are fundamental features that users expect.

1. **Real-time Type Error Detection:** The tool provides instant feedback as developers write code, ensuring that errors are caught early.
2. **Integration with VSCode:** Users can access all its functionalities directly within the VSCode editor, without needing to switch tools.
3. **Hidden Runtime Error Prevention:** It will identify and resolve type-related issues that could otherwise cause crashes during runtime, ensuring more robust code.

4. **Feedback on Fixes:** Users can provide feedback on fix suggestions to help PyTypeWizard improve its accuracy and usability.

2.2.2 Expected Requirements

These features directly impact user satisfaction based on how well they are implemented.

1. **Context-aware solutions for detected type errors:** It will recommend type hints based on the code context to maintain consistent and accurate type usage across the project.
2. **Error Explanation and Learning:** It will recommend type hints based on the code context to maintain consistent and accurate type usage across the project.

2.2.3 Exciting Requirements

These features go beyond user expectations, providing unique value. Their presence can greatly enhance user satisfaction.

1. **LLM-Based Type Repair System:** Leveraging large language models to offer advanced, context-aware fixes introduces cutting-edge innovation that delights users.
2. **Code Context Incorporation using RAG:** Incorporating RAG to fetch relevant code context and generate accurate, context-specific type error fixes adds an intelligent layer to error handling, improving both the precision and relevance of suggestions.
3. **Integration of Feedback to Improve Code Context:** Collect user feedback to refine code context and enhance solution generation for future errors.

2.2 Usage Scenario

PyTypeWizard: An Intelligent Tool for Fixing Python Type Errors is a VSCode extension designed to streamline the process of identifying and resolving Python static type errors. It provides real-time error detection and context-aware fix suggestions, helping developers maintain code quality and avoid common type-related pitfalls. In addition to offering immediate solutions, PyTypeWizard explains each fix and guides preventing similar issues in future projects. This ensures developers can focus on writing efficient, error-free code with minimal interruptions. The usage scenario for that particular tool is delineated below:

Installation

Users can download and install the PyTypeWizard extension from the VSCode Marketplace.

Real-time Error Detection

PyTypeWizard continuously analyzes Python code as developers work within the IDE. It detects static type errors based on Python's type hints and runtime type analysis. The extension uses a machine learning model trained on common type errors to provide accurate and context-specific error identification.

Real-time Fix Suggestions and Explanations

When a type error is detected, PyTypeWizard provides instant fix suggestions directly in the IDE. Users can view and apply the recommended fixes with a single click. Additionally, the extension explains each suggestion, helping developers understand why the error occurred and how the fix resolves it.

Educational Insights

PyTypeWizard includes guidance on best practices to avoid type-related errors in the future. This feature helps developers improve their coding habits and write more robust, type-safe code over time.

Feedback Mechanism for Fixes

Users can provide feedback on the accuracy of the suggested fixes. Whether a fix successfully resolved an error or introduced a new issue, developers can report their experience. PyTypeWizard leverages this feedback to improve its recommendation system, enhancing accuracy and usability.

3. Scenario-Based Modeling

Scenario-based modeling is a method used in software development and design to create models that illustrate how a system behaves or responds to different real-world situations or scenarios. These scenarios are usually based on user interactions, events, or conditions, helping developers visualize and understand how the software will function in various contexts. By simulating these scenarios, developers can identify potential issues, make informed design choices, and ensure that the software meets user needs in diverse usage situations.

3.1 Use Case Diagram

A use case diagram is a visual tool in software engineering that depicts how a system interacts with external entities or actors to accomplish specific tasks or goals. It shows the connections between use cases (which represent system functions) and actors (which represent external users or systems). Use case diagrams are typically created during the requirements gathering phase, where system functionalities and user interactions are identified and then mapped out using specialized modeling tools.

3.1.1 Level 0:

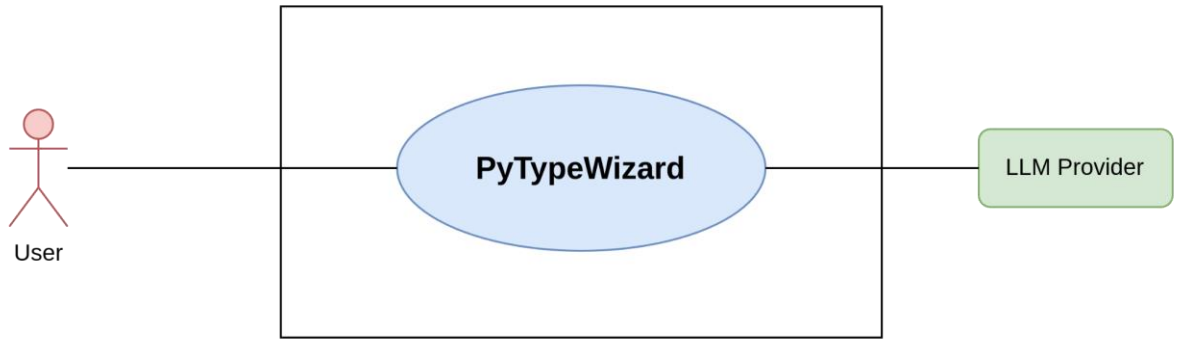


Figure 3: *Use Case Level 0 (PyTypeWizard)*

Name: PyTypeWizard System Overview
Primary Actors: User
Secondary Actors: LLM Provider
Goal in Context: The diagram above illustrates the complete system overview for PyTypeWizard, a tool designed for detecting and fixing Python static type errors.

3.1.1 Level 1:

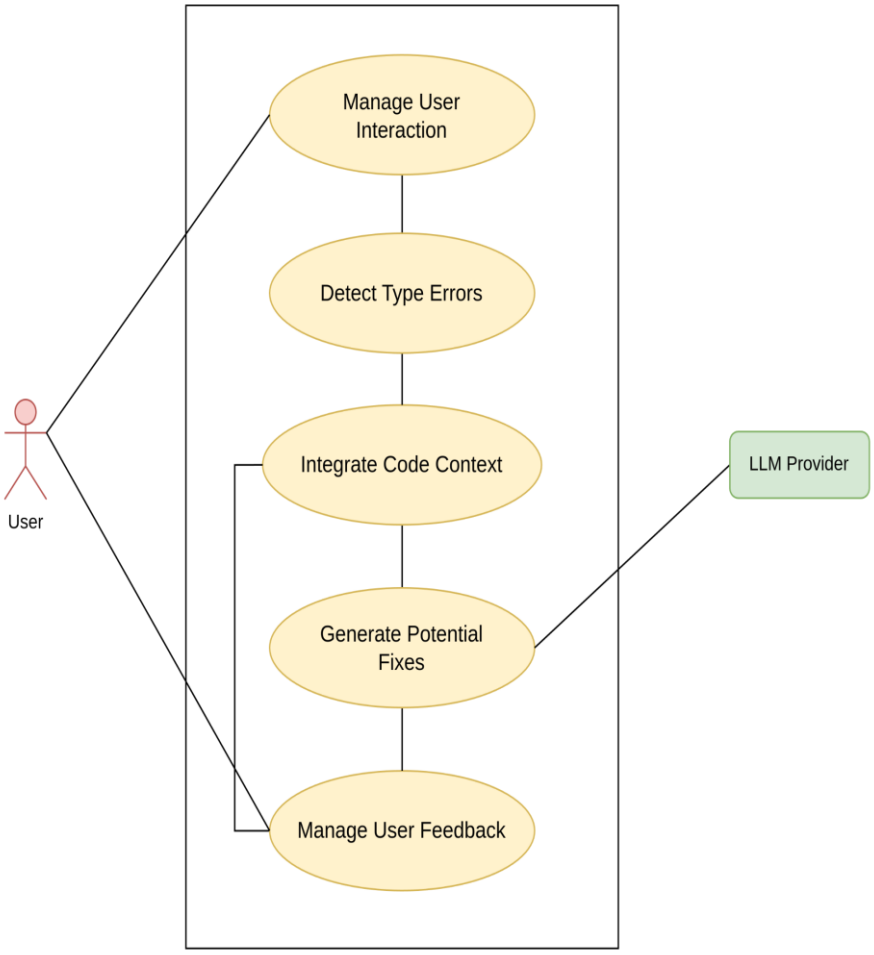


Figure 4: *Use Case Level 1 (Modules of PyTypeWizard)*

Name: Modules of PyTypeWizard
Primary Actors: User
Secondary Actors: LLM Provider
Goal in Context: The diagram above shows the modules of PyTypeWizard. It consists of 5 modules which are further elaborated below:

1. Manage User Interaction

- **Functionality**: This module activates when a user interacts with PyTypeWizard, typically by opening a Python file or initiating a type-checking operation within VSCode.
- **Purpose**: It ensures that the system components are ready to perform their tasks, initializing processes like error detection, code context integration, and fixing suggestion workflows.
- **Key Role**: Acts as the starting point for all system operations.

2. Detect Type Errors

- **Functionality**: Scans the user's code for static type errors, such as mismatched types, missing annotations, or improper usage of type hints.
- **Purpose**: Prevents hidden runtime errors by proactively identifying type-related issues during development.
- **Key Dependencies**: Uses the **LLM** and pre-configured rules to improve error detection accuracy.
- **Output**: Highlights the detected errors in real-time, providing immediate feedback to the user.

3. Integrate Code Context

- **Functionality**: Gathers relevant code snippets or surrounding code context from the user's project using RAG (Retrieval Augmented Generation). This is crucial for providing context-aware type suggestions.
- **Purpose**: By understanding the broader code context (e.g., variable types, function definitions, imports), this module enables more accurate and intelligent fixes.
- **Key Dependencies**: Leveraging the power of **LLM** and integrating the code context it ensures all suggestions are tailored to the specific code environment.

4. Generate Potential Fixes

- **Functionality**: Generates intelligent solutions for the detected type errors, offering fixes that align with the specific coding context.
- **Purpose**: Minimizes the manual effort required to resolve type errors by suggesting precise corrections.
- **Key Dependencies**: Using the **LLM** along with following RAG technique it ensures the suggested fixes are contextually accurate and aligned with best practices.

5. Manage User Feedback

- **Functionality**: Collects feedback from users about the quality and relevance of the suggested fixes. This could include confirming the fix, rejecting it, or providing custom corrections.
- **Purpose**: Improves the system's accuracy over time by incorporating real-world user feedback into its learning loop.
- **Key Role**: Ensures continuous improvement of potential solutions, making the tool more reliable and user-friendly.

4. Data Based Modeling

Database modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

The entities of PyTypeWizard after analyzing the usage scenario are:

- Solutions
- Chunks
- Repositories

The ER-Diagram of PyTypeWizard can be shown as below ER-Diagram of PyTypeWizard can be shown as below:

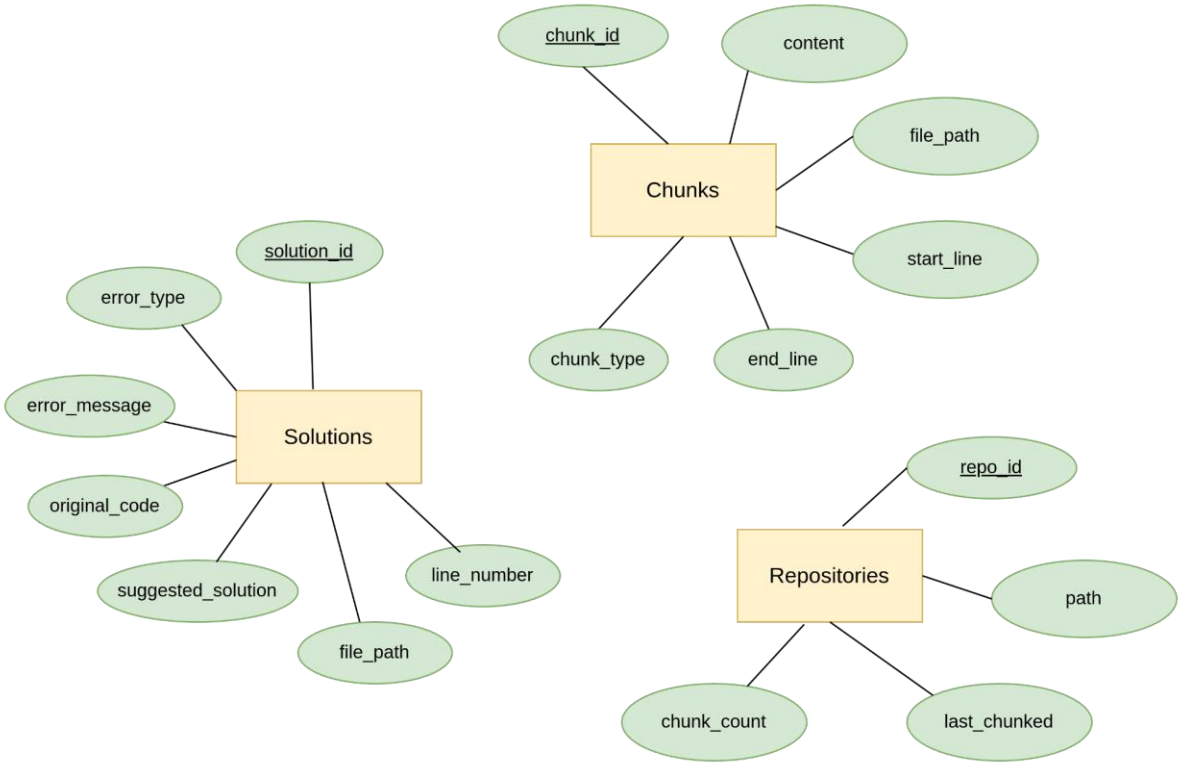


Figure 5: *ER Diagram of PyTypeWizard*

The schemas derived from the ER diagram is as follows:

Schema	**Attributes**	**Type**
Solution	id (PK)	VARCHAR(255)
	error_type	VARCHAR(100)
	error_message	TEXT
	original_code	TEXT
	suggested_solution	TEXT
	file_path	VARCHAR(255)
	line_number	INTEGER
Code_Files	id (PK)	VARCHAR(255)
	content (FK)	TEXT
	file_path	VARCHAR(255)
	start_line	INTEGER
	end_line	INTEGER
	chunk_type	ENUM (‘function’, ‘standalone’)
Repositories	id (PK)	VARCHAR(255)
	path	TEXT
	last_chunked	DATETIME
	chunk_count	INTEGER

5. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes:

1. CodeContextAnalyzer
2. ErrorDetectionController
3. FixGenerationController
4. RAGAgent
5. VSCodeExtensionManager
6. FeedbackManagementController

5.2 Class Cards

CodeContextAnalyzer	
Attributes	**Methods**
- project_path - vector_db_path - embedding_model	+ load_documents() + split_documents() + generate_embeddings() + vectorize_code_snippet() + get_context()
Responsibilities	**Collaborators**
Analyze the codebase to extract contextual type information and index it for easy retrieval.	• RAGAgent • ErrorDetectionController

ErrorDetectionController	
Attributes	**Methods**
- detected_errors - error_types	+ highlight_errors() + get_error_details() + scan_file_changes()
Responsibilities	**Collaborators**
Monitor the code workspace for type-related issues and extract error details (type, location, message).	• CodeContextAnalyzer

FixGenerationController	
Attributes	**Methods**
- potential_fixes - transformer_model - model_hyper_parameters - code_context - user_feedbacks	+ generate_potential_fixes() + process_fix_suggestions()
Responsibilities	**Collaborators**
Generate and process potential fixes for type-related errors.	• RAGAgent • FeedbackManagementController

RAGAgent	
Attributes	**Methods**
- retrieval_model - code_corpus - vector_db_path	+ retrieve_context() + generate_explanation() + validate_generated_fix()
Responsibilities	**Collaborators**
Perform Retrieval-Augmented Generation to propose fixes using retrieved code context.	• CodeContextAnalyzer

VSCodeExtensionManager	
Attributes	**Methods**
- workspace_path - extension_configuration	+ activate_extension() + display_errors() + display_potential_fixes() + handle_user_feedback()
Responsibilities	**Collaborators**
Manage the user interface within VSCode, including displaying errors and suggestions.	• ErrorDetectionController • FeedbackManagementController

FeedbackManagementController	
Attributes	**Methods**
- feedback_data	+ record_user_feedback() + get_feedbacks() + process_feedback()
Responsibilities	**Collaborators**
Collect and process user feedback on suggested fixes to improve future recommendations.	• FixGenerationController • VSCodeExtensionManager

5.3 CRC Diagram

The CRC diagram of the classes is given below:

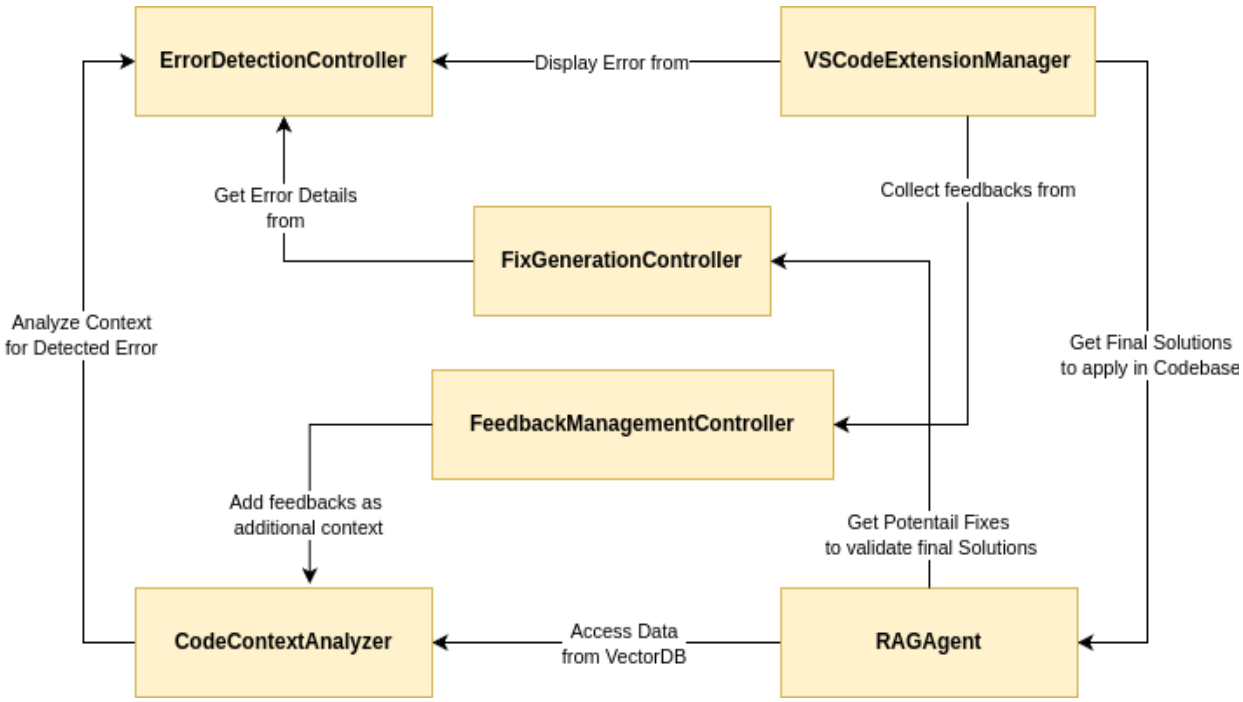


Figure 6: *CRC Diagram of PyTypeWizard*

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines a software system's high-level structure and organization. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

At the architectural design level, a software architect uses an architectural context diagram (ACD) to model how software interacts with entities
external to its boundaries. Systems that interoperate with the target system are represented as -

- **Superordinate systems**— These systems use the target system as part of some higher-level processing scheme.
- **Subordinate systems**— Those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.
- **Peer-level systems—** Those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system)
- **Actors—** Entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing.

Each of these external entities communicates with the target system through an Interface. Representation of PyTypeWizard in Architectural Context Diagram is delineated as follows:

- **Superordinate systems** - Our system does not have any superordinate system.
- **Subordinate systems** - Our system uses the *LLM* to generate potential fixes.
- **Peer-level systems** - Our system does not have any peer-level system.
- **Actors** - *Developers* are the core users of this system.

The architectural context diagram for the tool is given below-

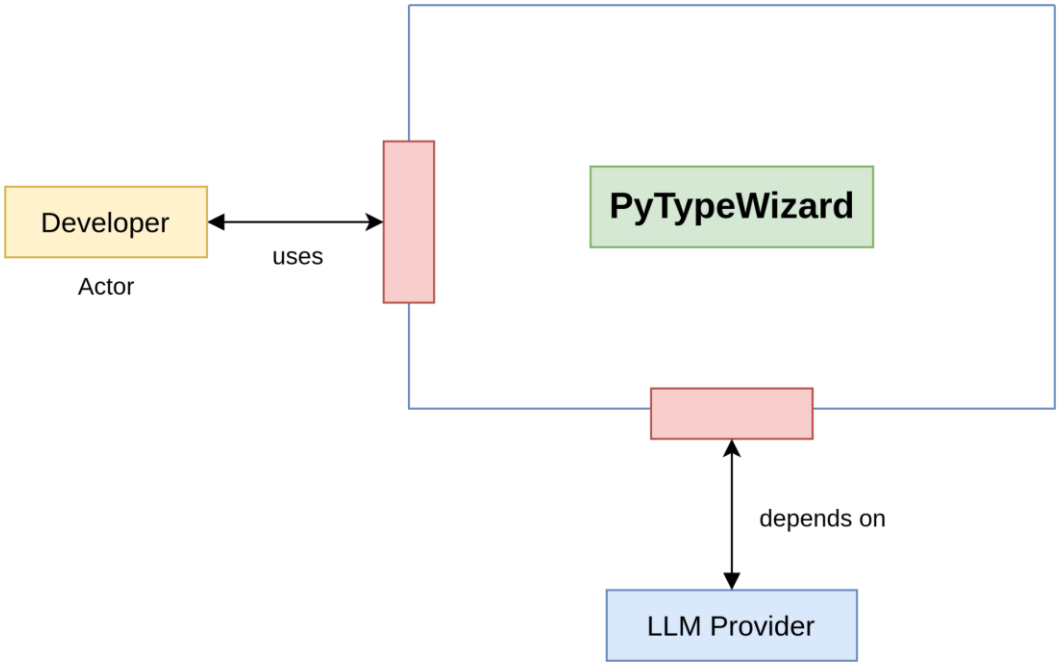


Figure 7: *Architectural Context Diagram*

6.2 Archetypes

The archetypes for the tool are defined below-

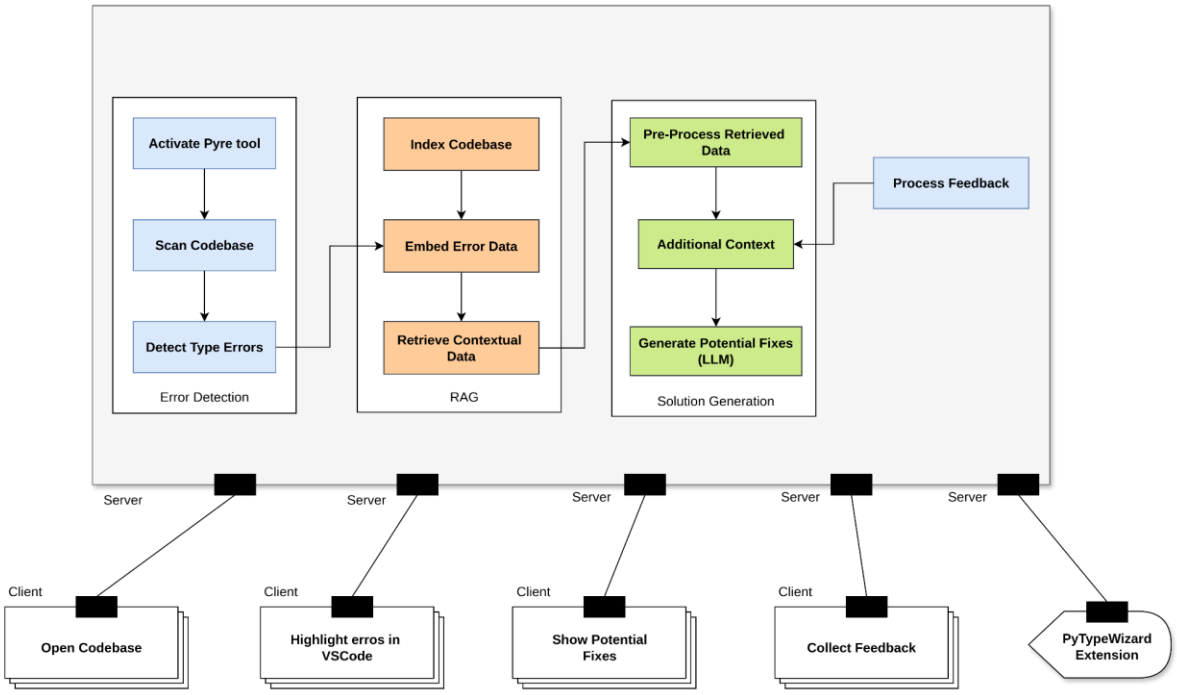


Figure 8: *Archetypes*

This client-server architecture is tailored to balance real-time performance with advanced processing capabilities. The **Error Detection** on the client side ensures that developers receive immediate feedback, minimizing workflow interruptions. The **RAG layer** and **Solution Generation** on the server-side offload computationally intensive tasks like context retrieval and LLM-based fix generation, making the system lightweight for the user while retaining high accuracy.

Error Detection (Client-side in VSCode)

- **Activate Pyre Tool**:
  - Triggers the Pyre type-checker upon opening a Python project in VSCode.
  - Acts as the initial stage to identify type errors in real time.
  - Ensures fast, local error detection without dependency on external services.
- **Scan Codebase**:
  - Iterates through the project files to gather the complete code structure.
  - Prepares the codebase for indexing and contextual analysis.
  - Efficiently detects type-related issues and marks them for review.
- **Detect Type Errors**:
  - Highlights errors with red squiggles in the editor for immediate visibility.
  - Provides developers with a quick overview of unresolved issues.

RAG (Retrieval-Augmented Generation)

- **Index Codebase**:
  - Creates a vectorized representation of the project's codebase for efficient retrieval.
  - Uses embeddings to map all functions, classes, and their contexts.
- **Embed Error Data**:
  - Processes the detected errors and links them with the indexed codebase.
  - Prepares the error context for retrieval and fix generation.
- **Retrieve Contextual Data**:
  - Retrieves relevant parts of the codebase based on the error context.
  - Ensures that solutions are generated with complete awareness of the surrounding code.

Solution Generation (Server-side)

- **Pre-process Retrieved Data**:
  - Refines the retrieved code snippets for use by the LLM.
  - Ensures that unnecessary or redundant data is filtered out before solution generation.
- **Additional Context**:
  - Enriches the error data with user feedback and historical fixes.
  - Incorporates feedback to generate more accurate and relevant solutions over time.
- **Generate Potential Fixes (LLM)**:
  - Uses a Transformer-based model to produce multiple solutions for the detected type error.
  - Leverages advanced language understanding to handle complex scenarios.
- **Validate Solutions (LLM)**:
  - The RAG agent ensures that the solutions are practical and syntactically correct.
  - Filters and ranks solutions based on their suitability for the provided context.

Feedback Loop

- **Process Feedback**:
  - Accepts user feedback on the accuracy of the applied fixes.
  - Logs feedback for future solution refinement and model improvement.
  - Send feedback as Additional Context in the Solution Generation segment.

6.3 Top Level Components

The overall architectural structure with top-level components is illustrated below-

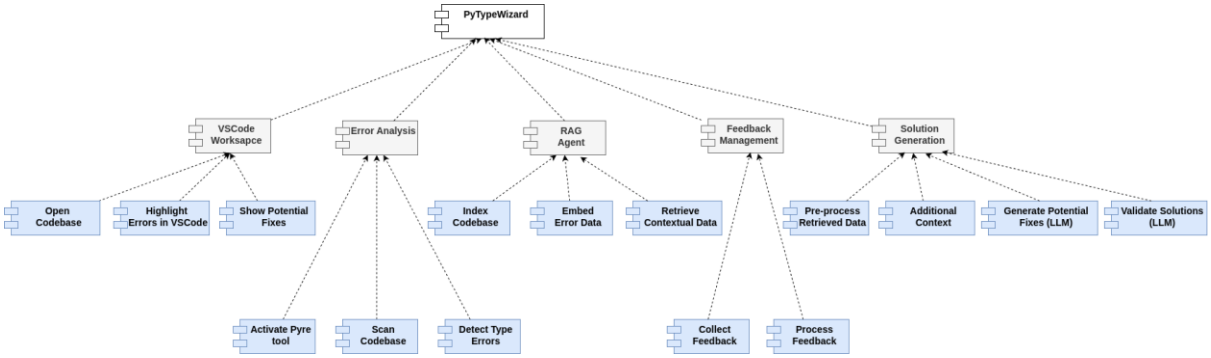


Figure 9: *Top Level Component*

6.4 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown below:

Requirements	**Components**
Real-time Type Error Detection	Activate Pyre Tool, Scan Codebase, Detect Type Errors, Highlight Errors in VSCode
Integration into VSCode	Open Codebase
Hidden Runtime Error Prevention	Scan Codebase, Detect Type Errors
Feedback on Fixes	Collect Feedback
Context-aware solutions for detected type errors.	Show Potential Fixes
Error Explanation and Learning	Display Fixes
LLM-Based Type Repair System	Validate Solutions, Generate Potential Fixes
Code Context Incorporation using RAG	Index Codebase, Embed Error Data, Retrieve Contextual Data, Pre-process Retrieved Data
Integration of Feedback to Improve Code Context	Additional Context, Process Feedback

9. User Interface Design

User interface design creates an effective communication medium between a human and a computer. Following a set of interface design principles, the design identifies interface objects and actions and then creates a screen layout that forms the basis for a user interface prototype.

Interface Analysis

In the case of user interface design, understanding the problem means understanding:

- The people (end users) who will interact with the system through the interface
- The tasks that end users must perform to do their work
- The content that is presented as part of the interface and
- The environment in which these tasks will be conducted

In the sections that follow, we examine each of these elements of interface analysis with the intent of establishing a solid foundation for the design tasks that follow.

9.1 User Analysis

All types of users will be able to use this tool. Mostly, software developers can get help from this tool to detect their Python Type Hint issues and fix them in runtime.

9.2 Task Analysis

9.2.1 User Task: Open VSCode Extension

- Launch **VSCode** and *open* the Extensions tab

9.2.2 User Task: Search PyTypeWizard & Install

- *Search* for **PyTypeWizard**
- *Click* install in the **PyTypeWizard Extension Page.**

9.2.3 User Task: Open Any Python Project

- *Opens* a **Python project** in VSCode

9.2.4 User Task: Open PyTypeWizard Sidebar

- *Click* on the **PyTypeWizard icon** to *open* the *sidebar*

9.2.5 User Task: View Runtime Type Errors

- *View* the detected **Type Errors** in runtime in the **IDE Text Editor**

9.2.6 User Task: Navigate to Error Location

- *Click* on **Error Type Label** to expand
- Click on the **Go To Button** to navigate the error location.

9.2.7 User Task: Generate Fix & Explanation

- *Hover* over any **Type Error**
- *Click* on **Quick Fix** option
- *Click* on **Fix & Explain** option

9.2.8 User Task: View Error Fix

- *View* the LLM-generated fix in the **Sidebar**

9.2.9 User Task: Ask PyTypeWizard for Further Explanation

- *Select* any **snippet or keyword**
- *Click* on the **Ask PyTypeWizard Button**

9.2.10 User Task: Regenerate Previous Fix

- *Click* on **Regenerate Response Button**

9.2.11 User Task: Save Generated Fix

- *Click* on **Save Solution button**

9.2.12 User Task: Apply Fix to Text Editor

- *Click* on **Apply Fix Button**
- *Click* on **Accept Button** to apply the change
- Click on **Reject Button** to discard the change

9.2.13 User Task: View Settings Page

- *Click* on the **Settings Icon**
- *View* the **Settings Page**

9.2.14 User Task: View History Page

- *Click* on the **History Icon**
- *View* the **History Page**

9.2.15 User Task: View About Type Hints Page

- *Click* on the **About Type Hints Tab**
- *View* the **About Type Hints** in the **Sidebar**

9.2.16 User Task: Ignore Error

- *Click* on the **Ignore this type error** option to ignore a single error.
- *Click on the* **Ignore all type error** *option to ignore all errors of a particular file.*

Objects (**boldface**) and actions (*italics*) are extracted from this list of tasks. This representation of objects (**boldface**) and actions (*italics*) are also maintained in the next section.

9.3 User Manual

User interface layouts with corresponding tasks, objects and actions and how to interact with them are illustrated below:

9.3.1 VSCode Extension Page

After launching the VSCode extension User will click on the Extension icon on the left panel (marked with red arrow). The extension Icon has been shown in the following figures.

Task 1: *Click* on the **Enable / Install button** (marked with red arrow) to install the extension in VSCode (Figure 10).

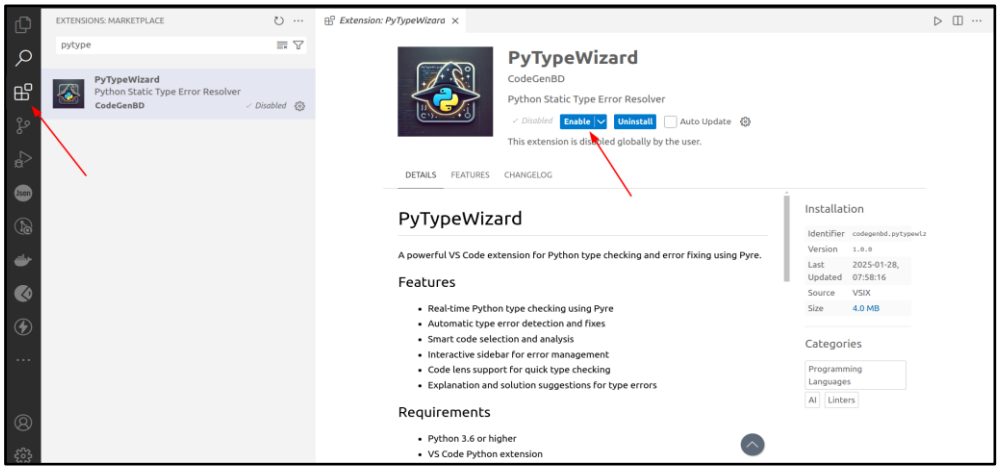


Figure 10: *VSCode Extension Page (with PyTypeWizard Extension)*

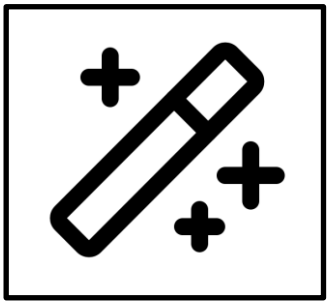


Figure 11: *PyTypeWizard Extension Icon*

Figure 12: *PyTypeWizard Extension Icon in VSCode [Activity Bar](#)*

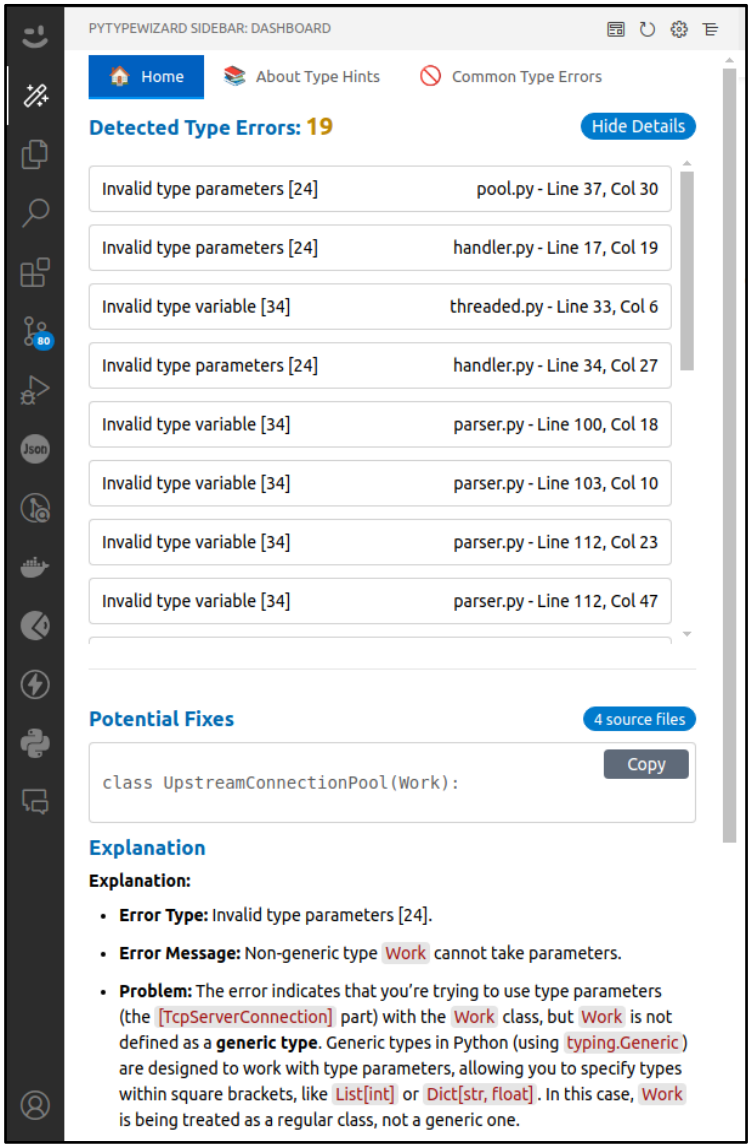


Figure 13: *PyTypeWizard Dashboard Page*

9.3.2 PyTypeWizard Dashboard

On the dashboard page, there are three parts namely:

1. Detected Error List
2. Potential Solution Section
3. Code Insights Section

Users can view the total number of errors in the header section. Then in the following list, users can see an overview of which errors have been detected along with the file and location name.

Task 2: The user *clicks* on the **Goto Button** (marked with a red arrow) which navigates to the exact location of the detected error in the **IDE** (Figure 14).

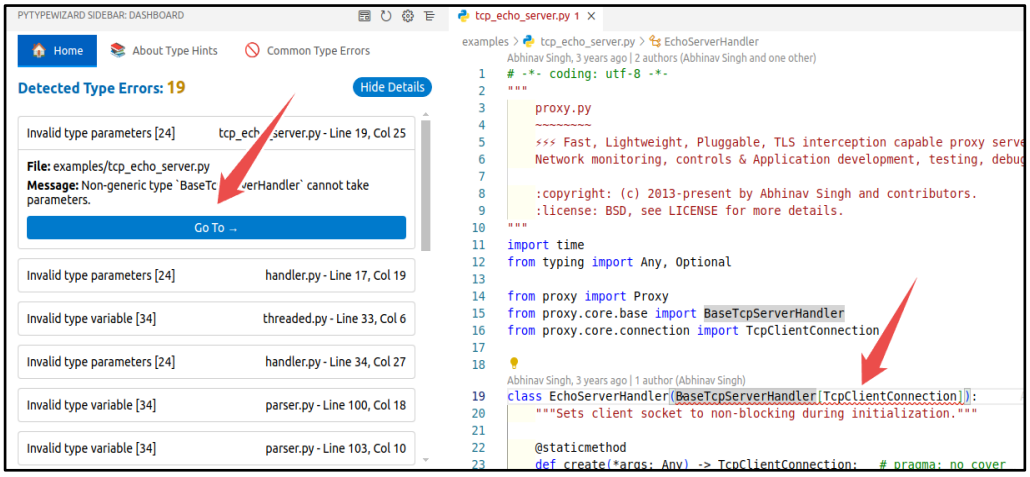


Figure 14: *Navigating Error Location from PyTypeWizard Home Page*

Task 3: Now the user will *hover* the mouse over the **error line** (red squiggle) and if necessary, then *scroll down* a bit to find the Quick Fix Button (Figure 15).

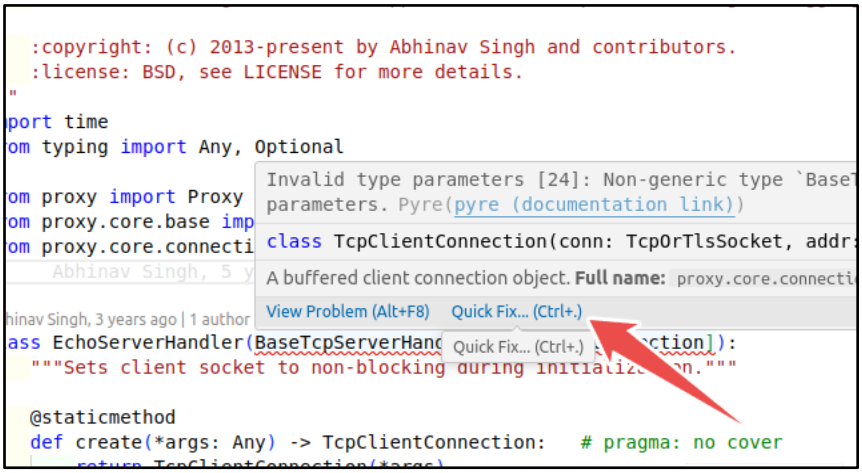


Figure 15: *Hover over the error line & find the Quick Fix button*

Task 4: After *clicking* on the **Quick Fix Button** user will get the following options shown in figure (Figure 15).

Task 5: Users will *click* on the **Fix & Explain Button** which will generate a potential fix along with an explanation in the **Sidebar** (Figure 16).

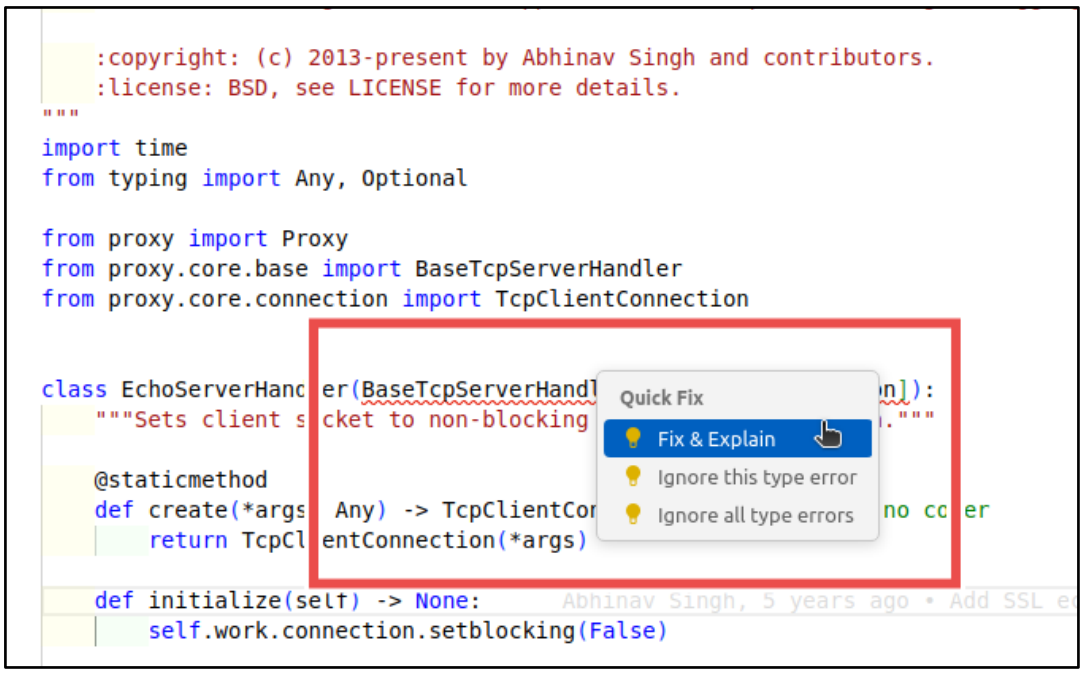


Figure 16: *Clicking on the Fix & Explain option*

Task 6: Users can *view* the generated potential solution in the **PyTypeWizard Dashboard** (Figure 17).

In the sidebar, where the potential solution is displayed, we can notice two segments. The upper red segment of the image (Figure 17) shows the script's name and count of which **context** is used to generate the solution. The list can be expanded or collapsed by *clicking* on the **counter (3 source files).**

Next, within the yellow portion (Figure 17), the solution has been explained and the potentially correct code is shown in a box. Users can *click* on the **Copy button** to copy the snippet into the clipboard.

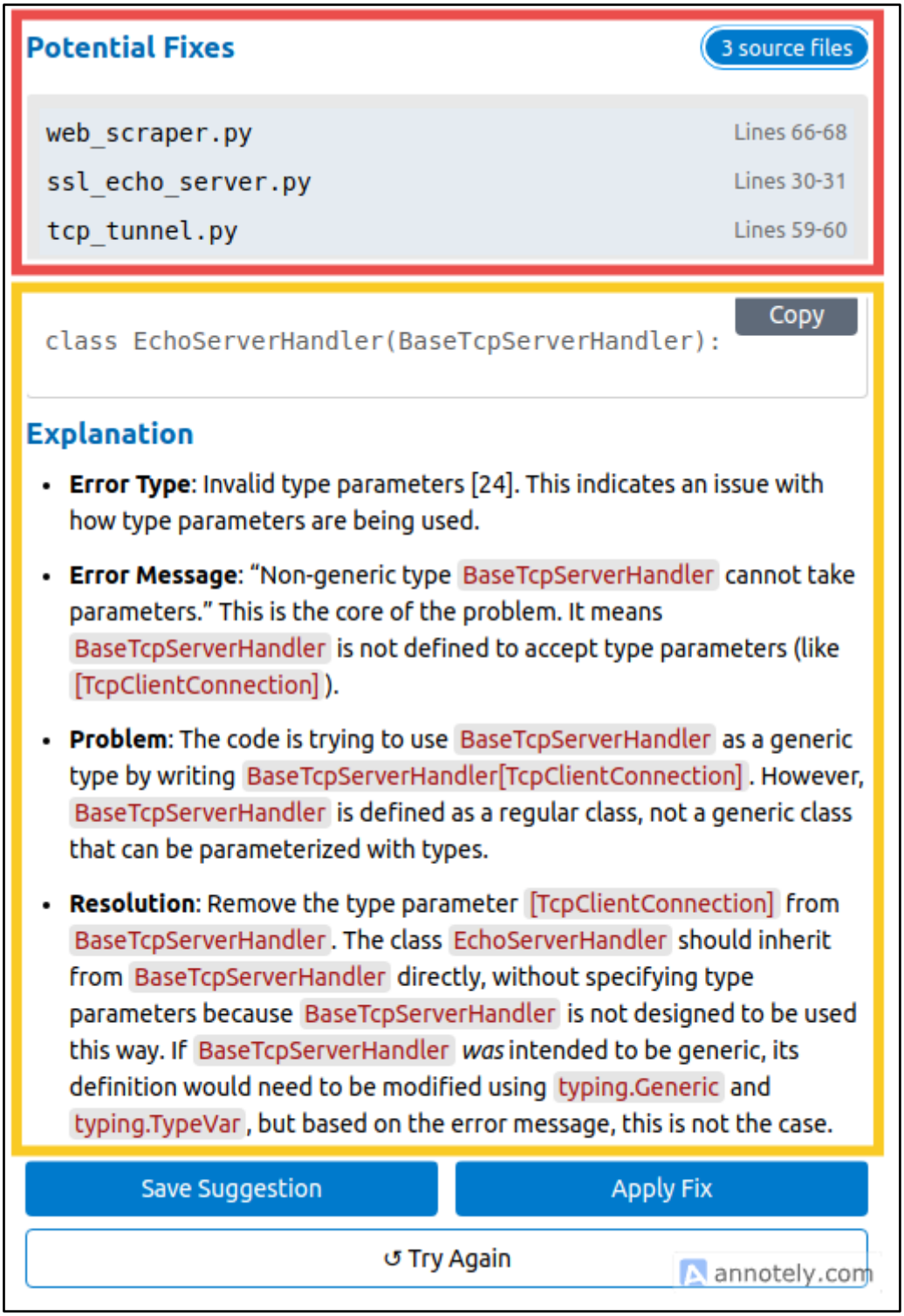


Figure 17: *Potential Solution Generated by LLM*

Task 7: To save this solution for future reference users have to *click* on the **Save Suggestion Button** which will pop up a notification mentioning that it has been saved (Figure 18).

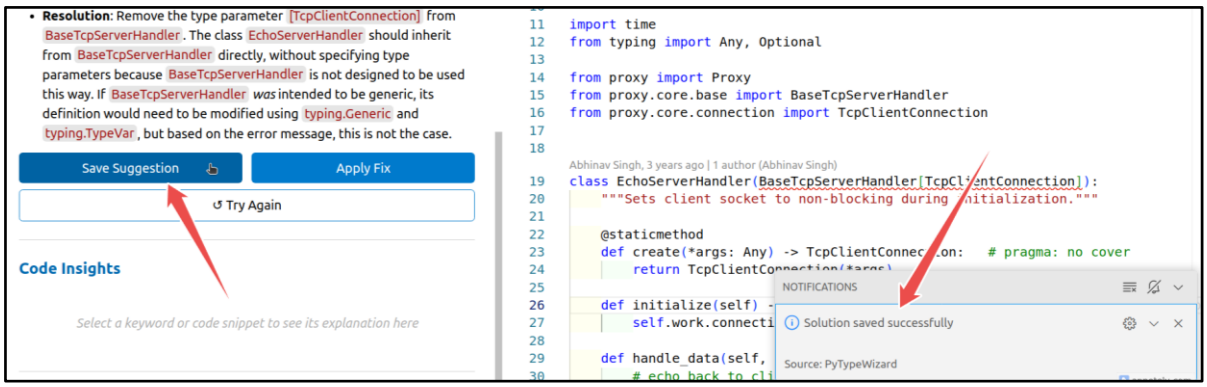


Figure 18: *Saving Generated Solution for Future Reference*

Task 8: To apply the generated code in the actual codebase, the user will *click* on the **Apply Fix Button** and eventually the fixed code will be placed on the editor (Figure 19).

Task 9: If the user *clicks* on the Accept **Fix Button,** then it will be added to the editor. On the other hand, if he *clicks* on **Reject Fix Button** the selected code will be removed from the editor (Figure 19).

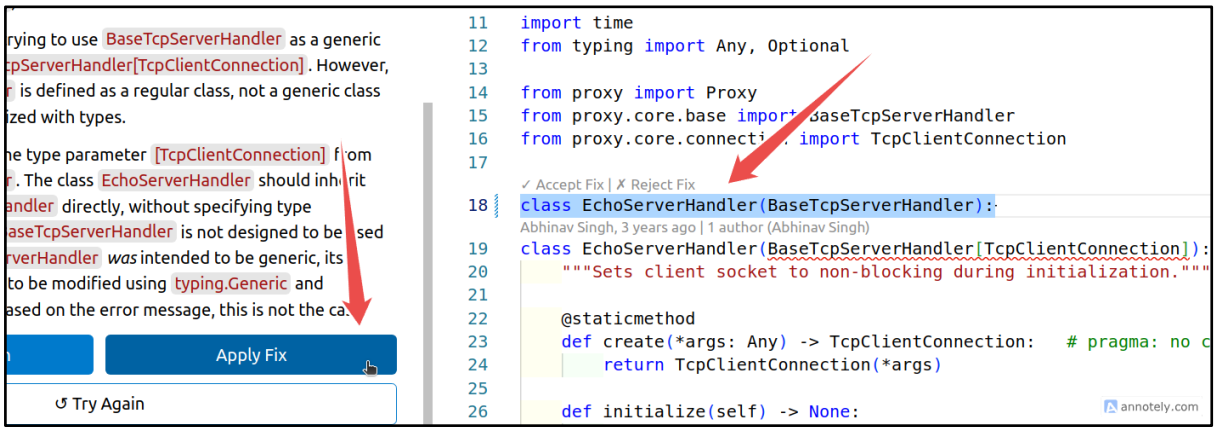


Figure 19: *Applying Generated Fix Directly on the Editor.*

Task 10: If the fix seems wrong or irrelevant, the user can *click* on the **Try Again Button** to re-generate another response based on the previous response and other details (Figure 20).

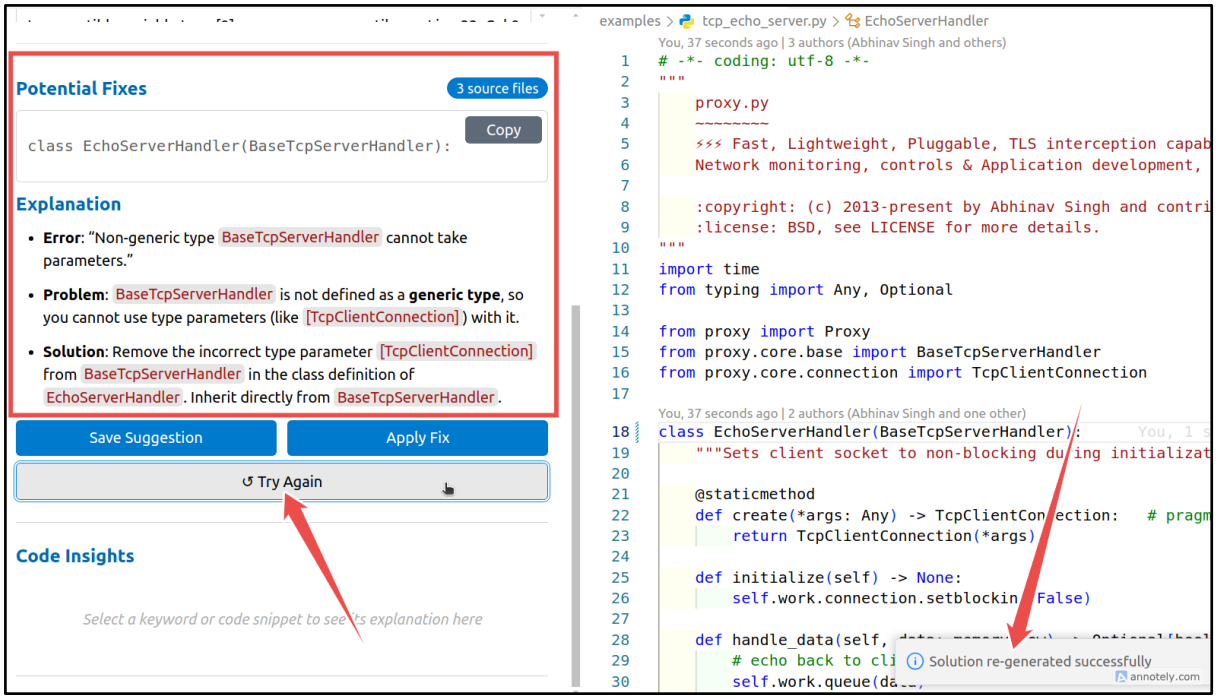


Figure 20: *Generated another fix for the detected error*

When the error is resolved the detected count on the PyTypeWizard Dashboard is updated in real time and the error line (red squiggle) is also removed (Figure 21).

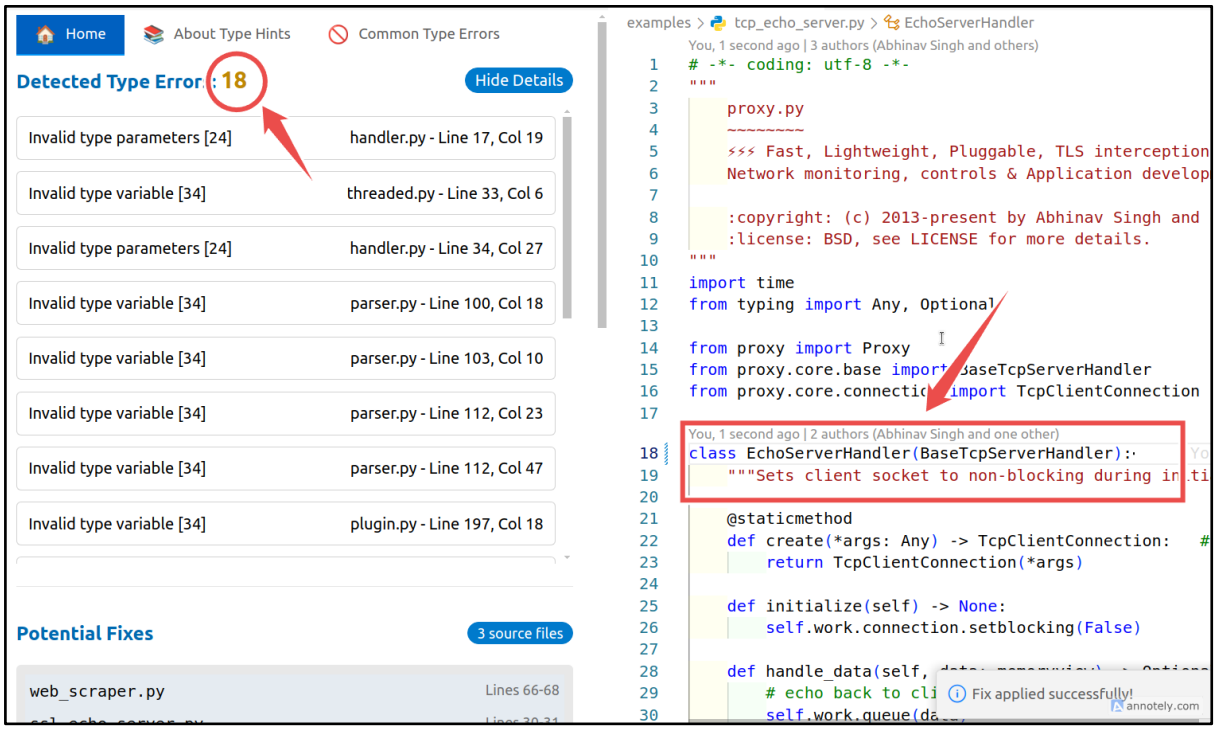


Figure 21: *Error resolved and error count updated*

Sometimes the user needs to ignore any particular error or skip all the errors of a particular file. For that scenario, we have added an intuitive ignore error feature

Task 11: Users will *hover* over the **error line** (red squiggle) and *click* on the **Quick Fix Button.** Then based on the user preference he can click either **ignore this type error** or **ignore all type error** option (Figure 22).

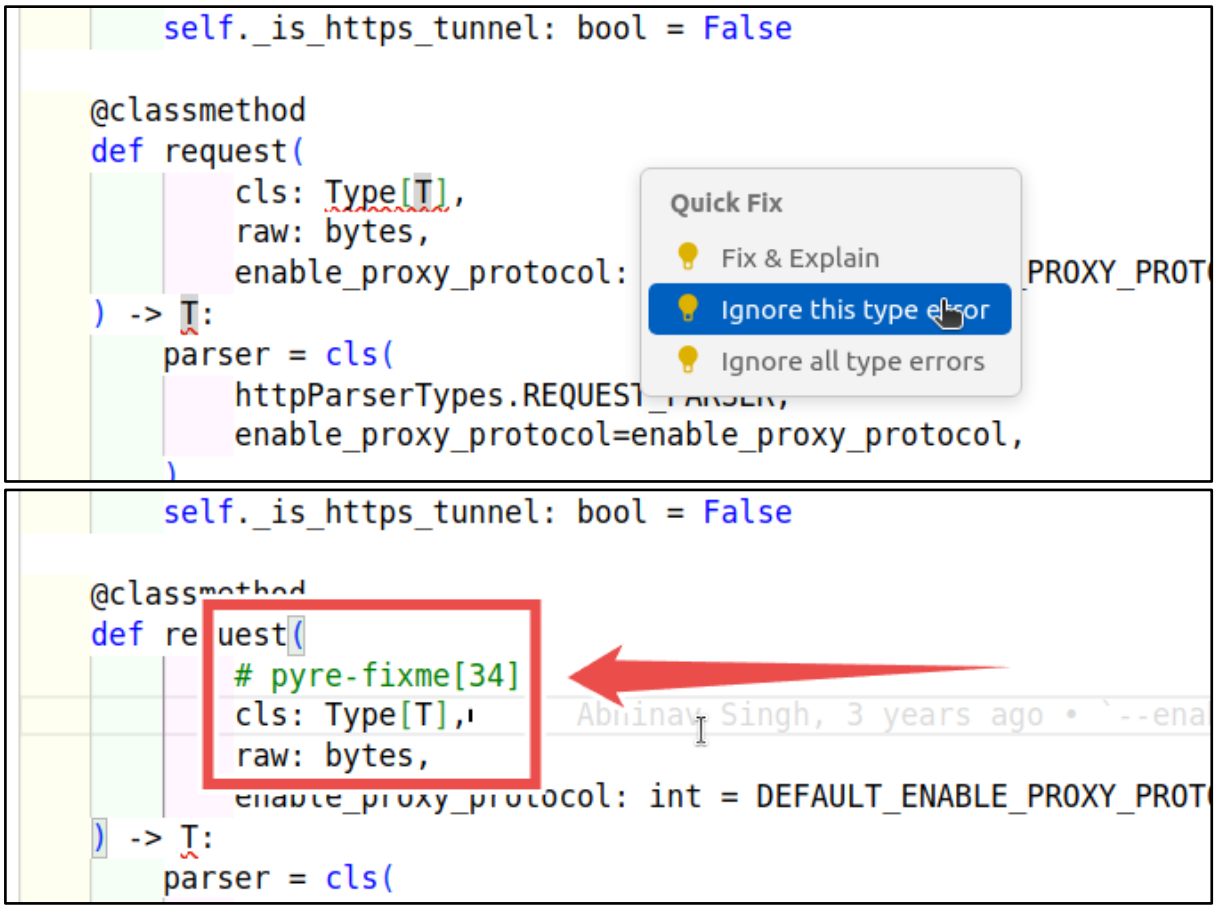


Figure 22: *Ignore any particular error*

Now let's learn about our next segment **Code Insight.** If a user wants to learn more about any particular keyword or code snippet, he can use the Ask PyTypeWizard feature.

Task 12: Users *select* any particular **keyword/snippet** and then *click* on the **Ask PyTypeWizard Button.** Users will be asked to *provide* any **prompt** regarding what they want to know. Then based on the prompt it will generate a suitable response in **PyTypeWizard Dashboard.** (Figure 23)

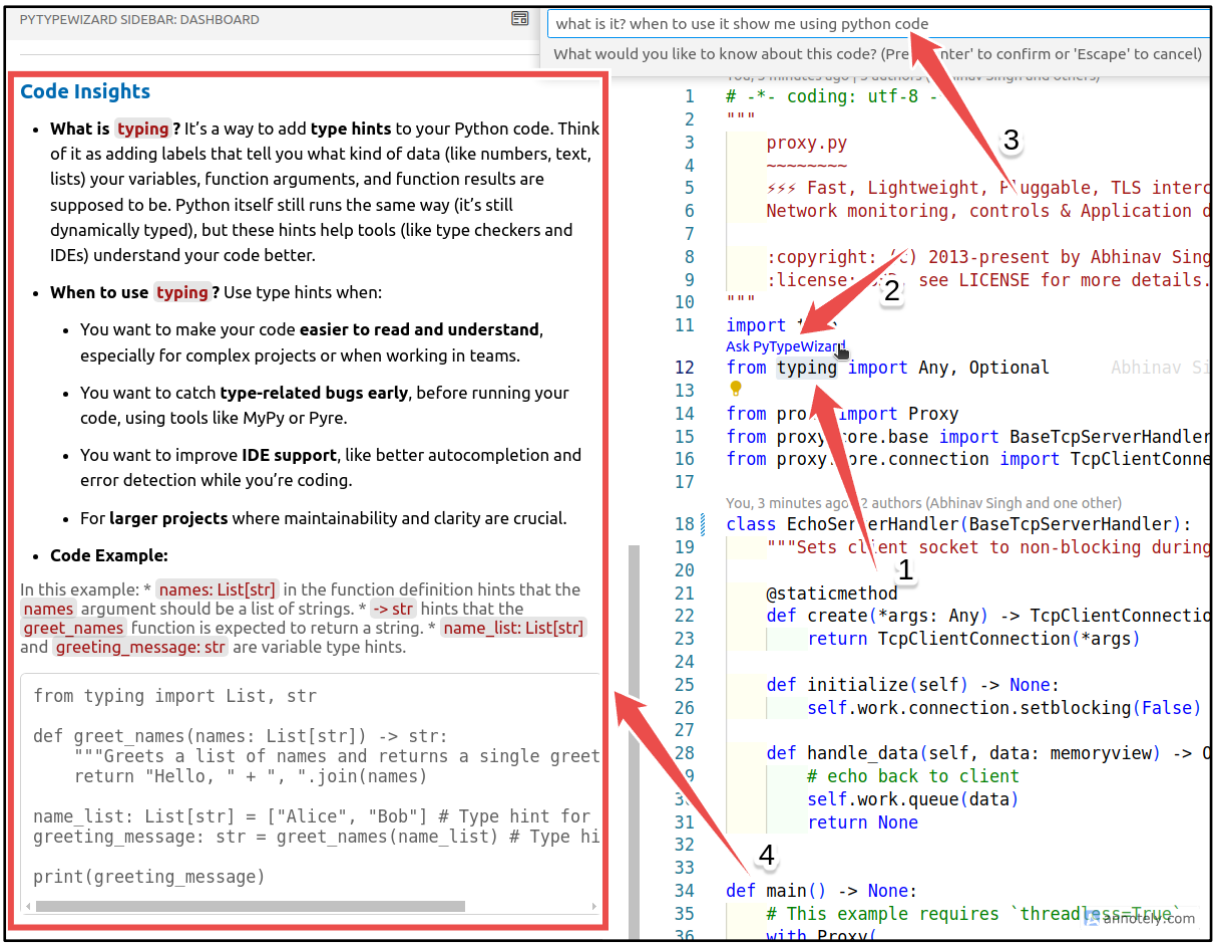


Figure 23: *Ask PyTypeWizard Feature Working Process*

9.3.3 About Type Hints Page

To acknowledge the user about the basic and advanced Python Type Hints I have added a detailed page of Python Type Hints.

Task 13: Users have to *click* on the **About Type Hints Tab** to navigate to that particular page (Figure 24).

Here users can learn more about the typing convention of Python. In which scenario we should use suitable Python Types to enhance the code readability and maintainability? Users can also navigate to desired Type hints details by *clicking* on the **items** in the Quick Navigation section.

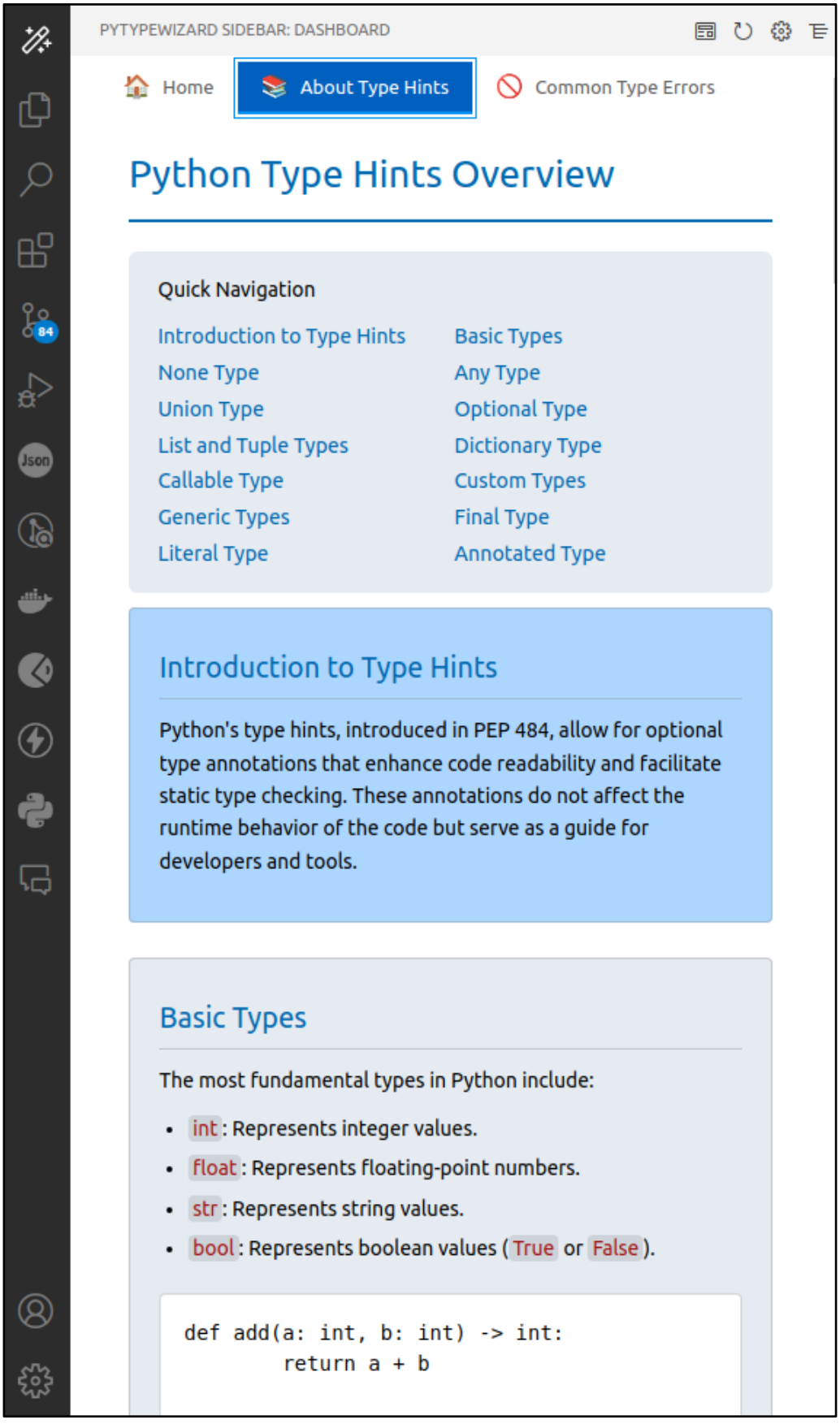


Figure 24: *About Type Hints page view*

9.3.4 Common Type Errors Page

To acknowledge the common typing issues of Python Hints, we have a dedicated page Common Type Errors. Here users can find the commonly occurring type errors, reasons, and dos and don'ts to avoid those errors.

Task 14: Users have to *click* on the **Common Type Errors** to navigate to that particular page (Figure 25).

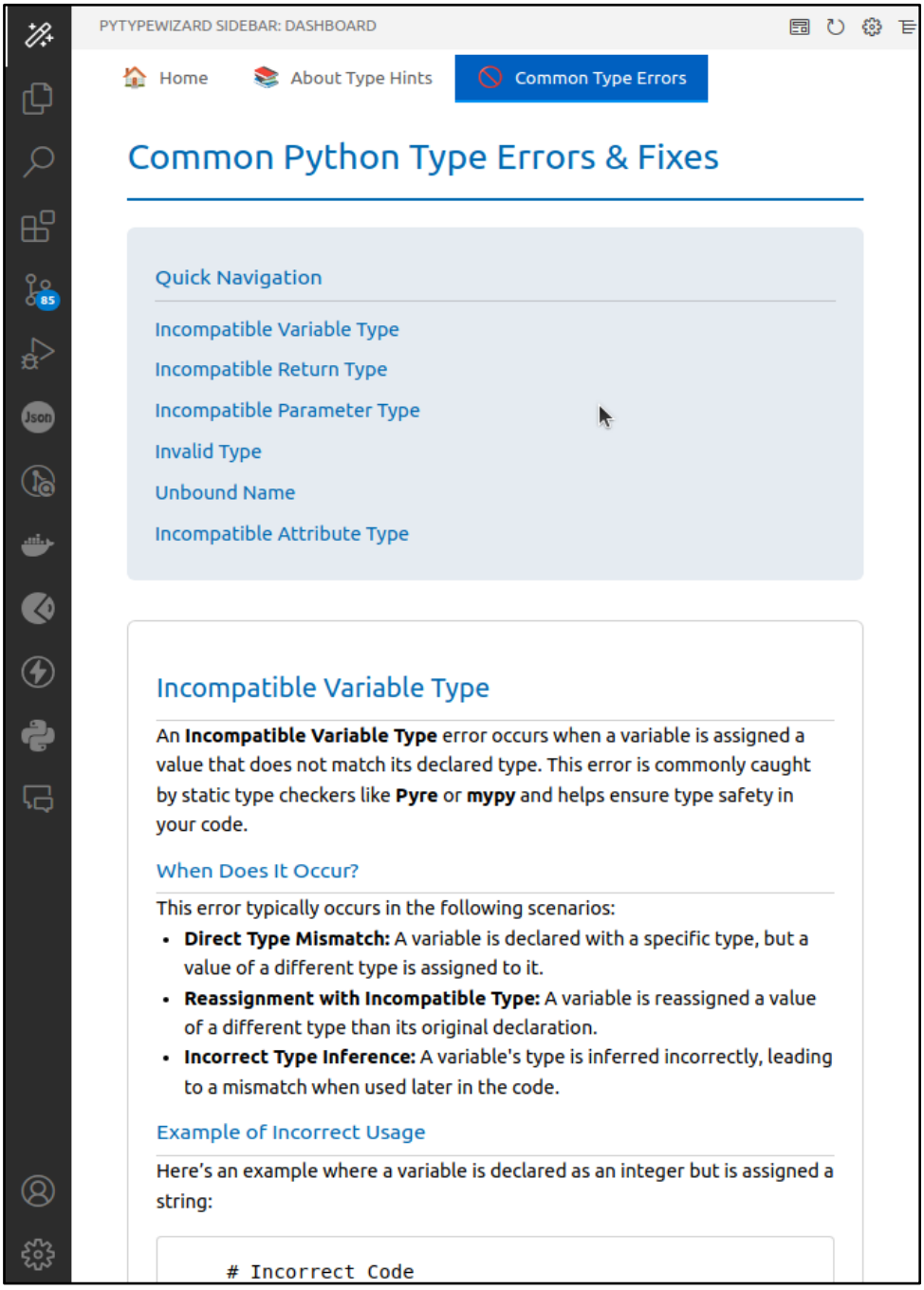


Figure 25: *Common Type Errors page view*

9.3.5 Previous Fix Record Page

To view our saved suggestions (Task 12), we have a dedicated page namely **Previous Fix Records** where users can view and search the details of the previously saved records.

Task 15: Users have to *click* on the **History Icon** of the Sidebar Toolbar on top and it will navigate to the **Previous Fix Record Page** (Figure 26)**.**

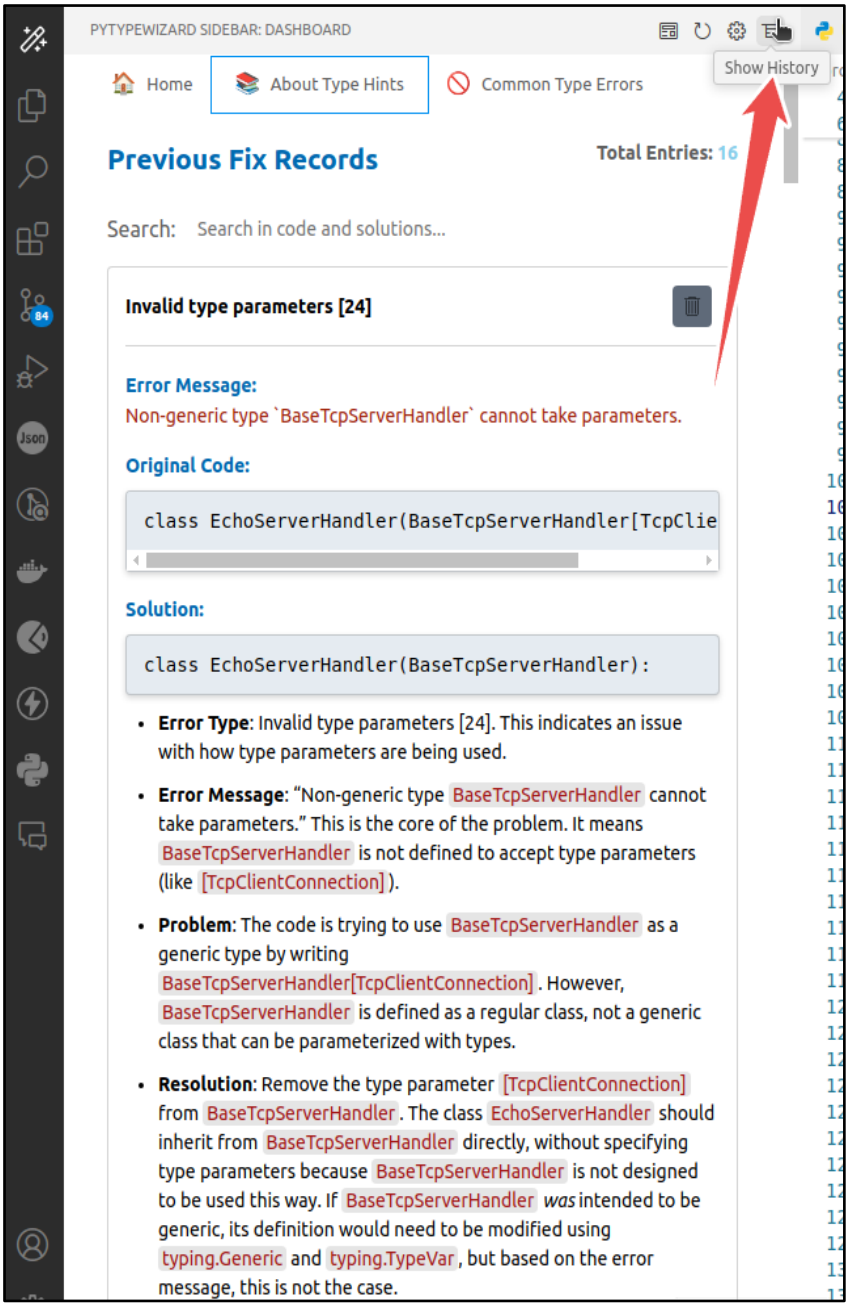


Figure 26: *Previous fix records page view*

9.3.6 Settings Page

To Customize some parameters of this application we also have a dedicated settings page.

Task 15: Users have to *click* on the **Settings Icon** of the Sidebar Toolbar on top and it will navigate to the **Settings Page** (Figure 27)**.**

Here user can configure a couple of features including:

- *Enable CodeLens:* It defines the enable/disable option for the **Ask PyTypeWizard** feature.
- *Enable Error Types:* Here the user can select which types of errors he wants to be detected.
- *API Key:* Users will put their Gemini API key in this text box.
- *LLM Provider:* Users can select suitable LLM providers from the list.

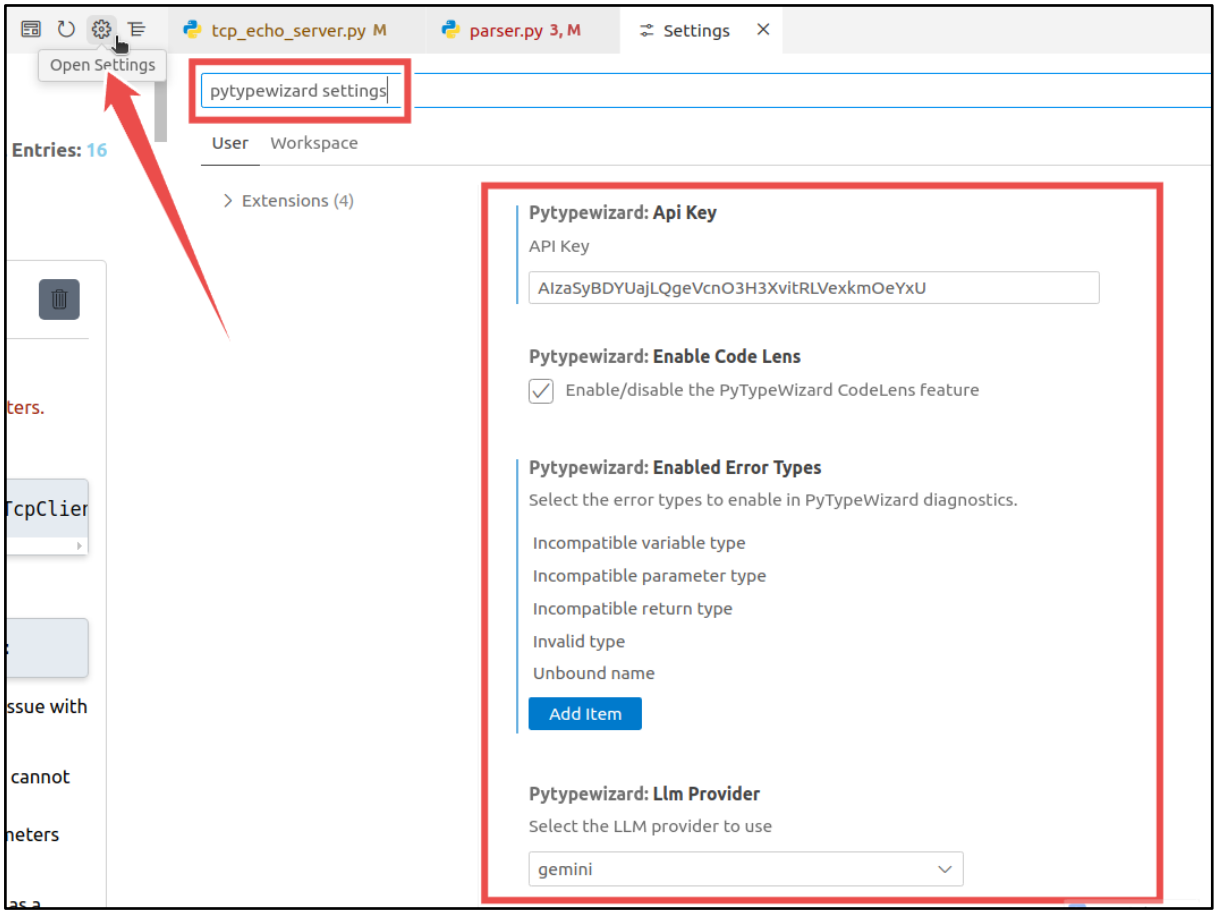


Figure 27: *PyTypeWizard Settings Page*

9.3.7 Clear LLM Conversation History

We incorporated relevant context to detect errors to assist the LLM in generating accurate fixes. Sometimes when encountering the situation, the LLM responses are quite similar even if we Retry again and again. In such cases, we have to clear the existing conversation history.

Task 16: Users have to *click* on the **Reset Icon** of the Sidebar Toolbar on top and it will reset the Conversation History of LLM (Figure 28)**.**

Then you should again perform **Task 5** to incorporate the context for generating fixes.

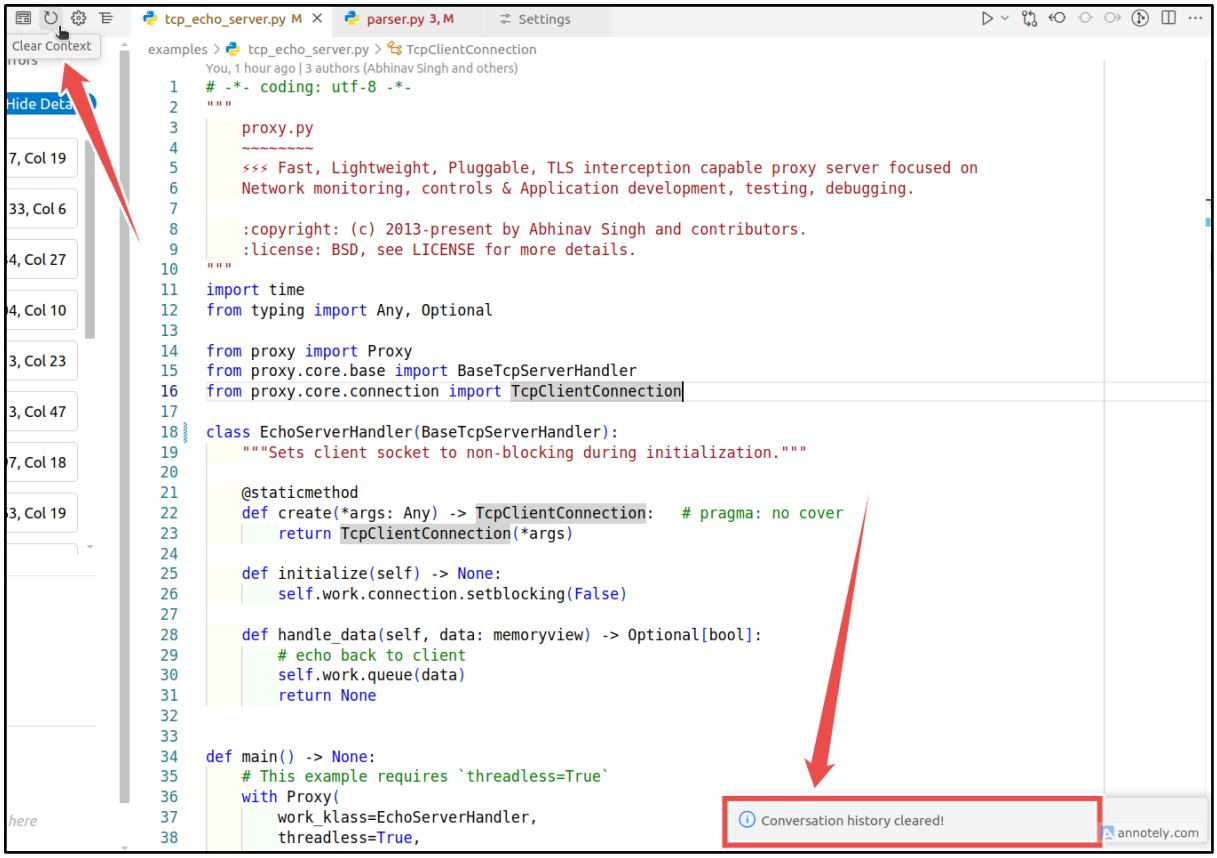


Figure 28: *Clearing LLM Conversation History*

9.3.8 Indexing Codebase

When we start with a new project, we need to divide the project files into chunks which is commonly known as indexing. Those chunks will be used as additional knowledge or context while generating fixes for detected errors. As it's a bit heavy and time-consuming operation, we left the choice of indexing upon users.

Task 17: Users can *click* on the **leftmost icon of the Toolbar** to initiate the indexing process as shown in (Figure 29)

After initiating the process, it will list out possible Python files, divide them into suitable chunks, and store them in the local database for further use.

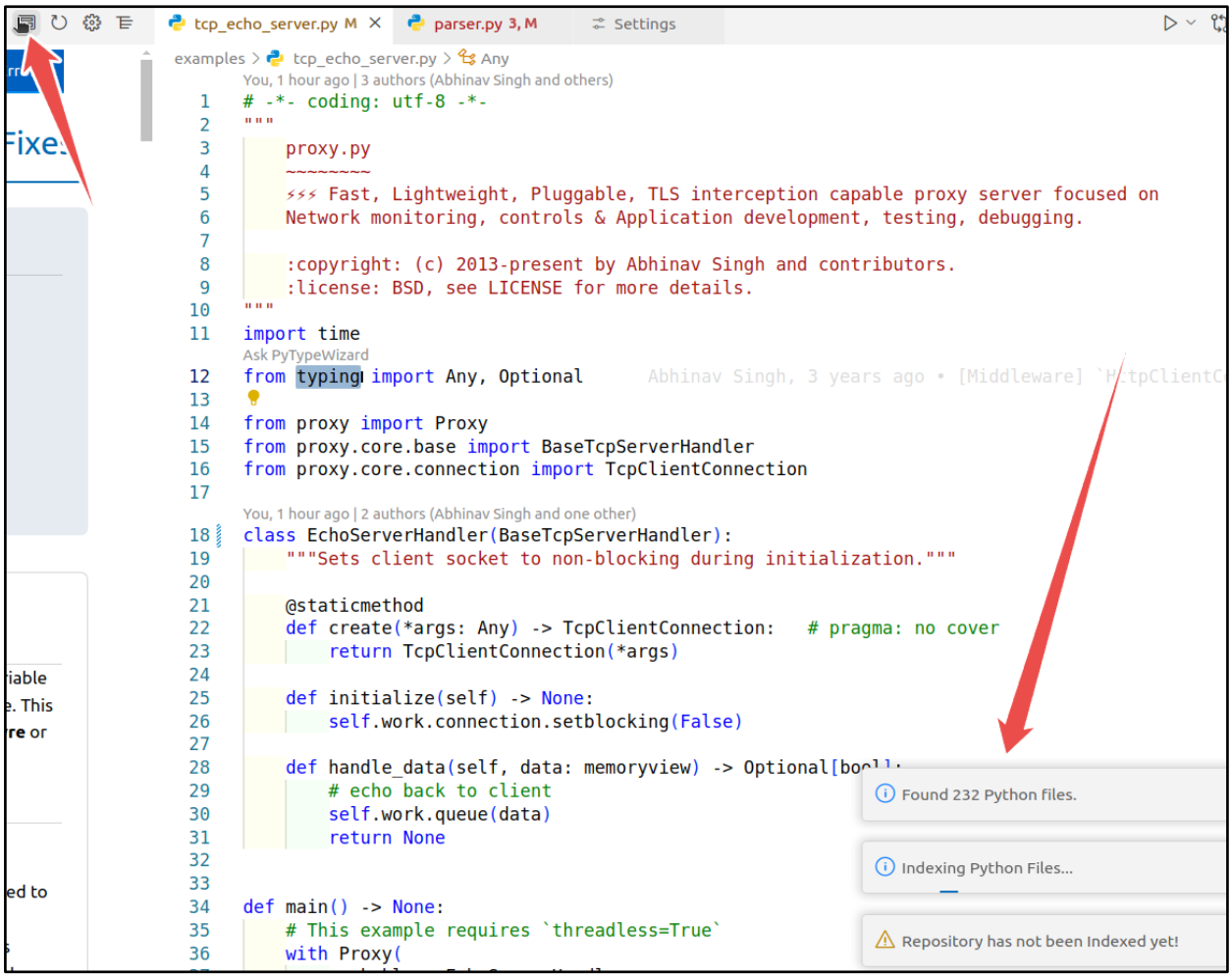


Figure 29: *Indexing active workspace files*

7. Preliminary Test Plan

In this chapter, a high-level description of testing goals and a summary of features to be tested are presented.

7.1 High-Level Description of Testing Goals

Testing goals for this tool are represented below from a high level-

- To verify that PyTypeWizard meets its most important functional and nonfunctional requirements.
- To ensure type-related error detection in real-time is correct and reliable.
- To ensure such type hint suggestions are contextually valid to provide better code quality.
- To reduce false positives and false negatives in error detection and fix suggestions.
- To confirm that the tool generates detailed and understandable error explanations.
- To test that user feedback on suggestions is collected, processed, and used effectively to enhance the tool.
- To guarantee seamless integration and smooth operation within the VSCode environment.

7.2 Test Cases

Test cases for the testing of this tool are documented below-

Test Case Id	T1
Title	Installation and Activation
Test Scenario	- Install the PyTypeWizard extension in a fresh VSCode installation. - Activate PyTypeWizard.
Expected Outcome	The extension installs without errors & activates successfully.
Pass/Fail	Passed

Test Case Id	T2
Title	Type Hint Checking

Test Scenario	- Write a function with some incorrect type hints, e.g., an incorrect return type annotation. - Verify that PyTypeWizard catches the invalid type hint.
Expected Outcome	PyTypeWizard flags the invalid type hint and draws a red squiggle under the error-prone line.
Pass/Fail	Passed

Test Case Id	T3
Title	Potential Solutions to Type Error
Test Scenario	- Request to Fix any detected error (an error where the parameter type is int instead of float) - Apply the suggested fix provided by PyTypeWizard.
Expected Outcome	The suggested fix accurately resolves the Type Error.
Pass/Fail	Passed

Test Case Id	T4
Title	Context-Aware Suggestions
Test Scenario	- Write a function with no type hints and ambiguous variable usage. - Request type hint suggestions for the function.
Expected Outcome	PyTypeWizard suggests accurate type hints based on the function's logic and context.
Pass/Fail	Passed

Test Case Id	T5
Title	Feedback Collection
Test Scenario	- Submit feedback on the generated potential solution (e.g., mark it as incorrect). - Check how PyTypeWizard processes the feedback.
Expected Outcome	- The feedback is successfully submitted along with a confirmation message

Pass/Fail	Passed

Test Case Id	T6
Title	Integration with LLM
Test Scenario	- Introduce a type error that requires contextual understanding. - Validate the fix suggestion provided by the LLM.
Expected Outcome	- PyTypeWizard provides a context-specific solution generated by the LLM. -The explanation includes reasoning derived from the code's context.
Pass/Fail	Passed

Test Case Id	T7
Title	Handling Custom Types
Test Scenario	Define custom types using TypedDict, NewType, or Union and introduce type errors involving these constructs.
Expected Outcome	- PyTypeWizard detects errors with custom types accurately. - Suggestions and fixes are generated for the custom type definitions.
Pass/Fail	Passed

Test Case Id	T8
Title	Check Context Extraction Capabilities
Test Scenario	Index the open codebase to extract relevant code context, including function definitions and variable types.
Expected Outcome	- The codebase is successfully indexed. - Relevant code context is extracted almost accurately, capturing necessary information for type error detection and solution generation.
Pass/Fail	Passed

Conclusion

In the current report, I have detailed all the necessary technical specifications required to develop the **PyTypeWizard**: an Automated Python Type Error Detection and Fix Suggestion Tool designed for VSCode. I have elaborated on the project scope, requirements, and usage scenarios in detail. Scenario-based modeling, data modeling, and class-based modeling were done to establish a resilient system architecture. In addition, a general overview of the goals of testing, as well as a summary of the features to be tested, has been presented. Diagrams and tables have been incorporated to ensure a thorough understanding of the technical aspects of this tool.

While PyTypeWizard can handle some of the fundamental issues of dynamic typing in Python by increasing the quality of code and reducing time spent on debugging it is not perfect. It might be further improved in many ways: by supporting more Python typing features, extending it to work with different IDEs, and continuing to use more advanced machine learning techniques to improve completion suggestions based on context.

This document should give a full understanding of the workflow and technical design of this tool. I hope it will be a helpful resource to gain insight into PyTypeWizard and its development process.

Reference

1. Chow, Yiu W., Luca Di Grazia, and Michael Pradel. "PyTy: Repairing Static Type Errors in Python." ArXiv, (2024). Accessed October 5, 2024. https://doi.org/10.1145/3597503.3639184.

2. Peng, Yun, et al. "Generative type inference for python." 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE). IEEE, 2023.